Formation Vue.js 3 · Jour 3 après-midi

Projet final : votre application Vue

Une application à vous, sur le sujet de votre choix, avec l'architecture vue pendant la formation. Le sujet est libre ; la structure est imposée.

DURÉE	TRAVAIL	RENDU	NOTE
13:30 – 16:55	**individuel**	**dépôt Git + démonstration**	**20 points + 4 de bonus**

1. Le déroulé

Heure	Étape	Ce qui doit être fait
13:30 – 13:45	**Fiche projet**	Choisir le sujet, remplir la fiche projet (dernière page), la faire valider par le formateur.
13:45 – 14:45	**Palier 1 : la base**	Projet, arborescence, routeur, store métier, service API, liste affichée depuis l'API.
14:45 – 15:15	**Palier 2 : les pages**	Détail, filtre, statistiques, 404, chargement différé, titres d'onglet.
15:15 – 15:30	Pause	
15:30 – 16:00	**Palier 2 : la connexion**	Store d'authentification, page de connexion, page protégée, build de production.
16:00 – 16:30	**Bonus**	Au choix, seulement si l'obligatoire fonctionne.
16:30 – 16:55	**Démonstrations**	Trois à cinq minutes par volontaire.

Un commit à la fin de chaque palier : « Palier 1 », « Palier 2 », puis « Bonus ». Terminez l'obligatoire avant de commencer un bonus : un projet simple qui fonctionne vaut mieux qu'un projet ambitieux cassé.

2. Choisir son sujet

Partez d'une ressource de l'API publique **DummyJSON** (`https://dummyjson.com`), gratuite et sans inscription. Les données sont en anglais : c'est accepté. Votre **interface** (titres, boutons, messages), elle, doit être en français.

Idée	Ressource	Clés utiles du JSON
Une boutique	`/products`	`title`, `price`, `category`, `stock`, `thumbnail`
Un carnet de recettes	`/recipes`	`name`, `difficulty`, `cookTimeMinutes`, `cuisine`, `image`
Un blog	`/posts`	`title`, `body`, `tags`, `views`
Un mur de citations	`/quotes`	`quote`, `author`
Un annuaire	`/users`	`firstName`, `lastName`, `email`, `image` (jamais `password`)

Avant d'écrire votre adaptateur, ouvrez la ressource dans le navigateur, par exemple `https://dummyjson.com/products/1`, pour voir les vraies clés. Une autre API publique sans clé est possible si le formateur la valide.

3. Le cahier des charges obligatoire

- [ ] **Arborescence** : les dossiers `components/common/` (au moins un composant générique `Base...` avec un slot), `components/<domaine>/`, `views/`, `router/`, `stores/`, `services/`.
- [ ] **Composants** : au moins 4 composants, avec des props, des événements (`emit`) et un `v-model` de composant (`defineModel`) pour un filtre ou une recherche.
- [ ] **Réactivité** : des propriétés calculées, et `onMounted` pour charger les données.
- [ ] **Routage** : 4 pages, la liste, le détail (`/xxx/:id`, avec l'identifiant converti en nombre), les statistiques et la page 404 ; le chargement différé ; le titre de l'onglet par `meta`.
- [ ] **Store métier** : un store Pinia (state, getters, actions), lu avec `storeToRefs`, avec une préférence sauvegardée par `persist`.
- [ ] **API** : les données viennent de l'API par un service Axios avec un adaptateur ; les états chargement, erreur et vide sont affichés.
- [ ] **Connexion** : un store d'authentification et une page de connexion (comme DevBoard, la version de démonstration suffit), et une page protégée par un guard.
- [ ] **Livraison** : toute l'interface en français ; `npm run build` puis `npm run preview` fonctionnent ; un commit par palier.

L'arborescence imposée

Structure du projet

```
mon-projet/
├── .env.development          VITE_API_URL=https://dummyjson.com
├── .env.production           la même variable, pour le build
└── src/
    ├── main.js               createApp(App).use(pinia).use(router)
    ├── App.vue               en-tête + <RouterView />
    ├── assets/
    ├── components/
    │   ├── common/           composants génériques (BaseXxx, en-tête)
    │   └── <domaine>/        composants du sujet (ProductCard, RecipeList...)
    ├── router/index.js       routes, chargement différé, 404, guard
    ├── services/api.js       Axios : instance, adaptateur, intercepteur
    ├── stores/               store métier + store d'authentification
    └── views/                une vue par page
```

4. Les bonus (un point chacun, quatre au maximum)

- [ ] Un **deuxième store métier** : favoris, panier, préférences…
- [ ] Une **route imbriquée** : une page avec des sous-pages et son propre `<RouterView />`.
- [ ] Des **transitions** : entre les pages, ou sur une liste avec `TransitionGroup`.
- [ ] Une **garde de sortie** (`onBeforeRouteLeave`) sur un formulaire.

- [ ] La **vraie connexion par jeton** : `POST /auth/login` de DummyJSON (compte `emilys` / `emilyspass`), intercepteur de requête, mot de passe jamais stocké.

5. Pour démarrer

Terminal

```
npm create vite@latest mon-projet -- --template vue
cd mon-projet
npm install
npm install vue-router@4 pinia pinia-plugin-persistedstate axios
git init
git add .
git commit -m "Initialisation"
npm run dev
```

.env.development

```
VITE_API_URL=https://dummyjson.com
```

Un exemple d'adaptateur, pour une boutique. Adaptez les clés à votre ressource :

src/services/api.js (exemple)

```
import axios from 'axios'

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 8000
})

// Adaptateur : les clés de l'API deviennent celles de votre application
const fromApi = (p) => ({
  id: p.id,
  nom: p.title,
  prix: p.price,
  categorie: p.category,
  enStock: p.stock > 0,
  image: p.thumbnail
})

const getProduits = async () => {
  // Axios décode le JSON : on sort seulement data
  const { data } = await apiClient.get('/products', { params: { limit: 20 } })
  return data.products.map(fromApi)
}

export default { getProduits }
```

Pour tout le reste (routeur, stores, connexion, page 404), repartez du code de **DevBoard** : chaque fichier du projet a son équivalent dans le dépôt de la formation.

6. La grille d'évaluation

Critère obligatoire	Points
Arborescence et composants (Base avec slot, props, emit, defineModel)	4
Réactivité (computed, onMounted)	2
Routage (liste, détail, statistiques, 404, chargement différé, titres)	4
Store métier (state, getters, actions, storeToRefs, préférence sauvegardée)	3
API : service, adaptateur, états chargement / erreur / vide	3
Store d'authentification et page protégée	2
Livraison : français, build de production, commits, démonstration	2
Total	**20**
Bonus (un point chacun)	+ 4 au maximum

7. Le rendu et la démonstration

- [ ] **Le dépôt Git** : les commits « Palier 1 », « Palier 2 » et « Bonus » ; le dossier `node_modules` et le dossier `dist/` ne sont jamais versionnés.
- [ ] **Le fichier `README.md`** : le nom et le but de l'application, la ressource de l'API, comment lancer le projet (`npm install`, `npm run dev`), et le compte de test.
- [ ] **La démonstration** (3 à 5 minutes) : lancer la version de production (`npm run preview`), montrer la liste, un détail, la page protégée et la page 404, puis un extrait de code choisi (le store ou le routeur).

8. Les erreurs fréquentes

- `undefined/products` dans l'onglet « Réseau » : la variable ne commence pas par `VITE_`, le fichier `.env.development` est mal nommé, ou `npm run dev` n'a pas été relancé.
- « Cannot read properties of undefined (reading 'map') » : `const data = await apiClient.get(...)` au lieu de `const { data } = ...`.
- L'écran ne se met plus à jour : des données du store déstructurées sans `storeToRefs`.
- Toutes les adresses mènent à la page 404 : la route `/:pathMatch(.*)*` n'est pas en dernier.
- La page de détail est toujours « introuvable » : l'identifiant de l'adresse n'est pas converti en nombre (`Number(route.params.id)`).
- Toujours « connecté » après la déconnexion : `logout()` doit remettre l'utilisateur **et** le jeton à zéro.
- Page blanche après `npm run build` : pas de fichier `.env.production`, ou `dist/index.html` ouvert directement au lieu de `npm run preview`.

Annexe : la fiche projet, à faire valider avant 13:45

À remplir dès le début de l'après-midi, puis à montrer au formateur avant de coder.

Rubrique	Votre réponse
Nom de l'application	
À quoi elle sert, en une phrase	
Ressource de l'API et champs gardés par l'adaptateur	
Page 1 : la liste, adresse	
Page 2 : le détail, adresse (avec `:id`)	
Page 3 : les statistiques, ce qu'elles comptent	
Page protégée (réservée aux connectés)	
Store métier : son état, ses getters, ses actions	
Préférence sauvegardée avec `persist`	
Filtre ou recherche avec `defineModel`	
Bonus visés	

Validation du formateur : .....